

Testing & Evaluating Probabilistic Systems

Tests, evals, golden scenarios, regressions, and how to separate model failures from product or software failures.

Core idea

Tests prove deterministic rules. Evals measure whether probabilistic behavior is good enough. A reliable AI product needs both.

Why AI quality needs two kinds of evidence

Layer	Best for	State example
Unit / contract test	Rules that should always hold	Rejected Evidence never mutates Current State; stale versions are blocked.
Integration test	Deterministic flows across components	Evidence -> interpretation record -> Review -> accepted transition -> History.
End-to-end test	User journeys across browser + API	Add a Note, review the change, confirm Project and History update.
LLM eval	Judgment where several outputs may be plausible	Did the model correctly decide whether new Evidence is consequential?
Manual / adversarial QA	Unexpected behavior and UX seams	Ambiguous notes, duplicate Reviews, weird navigation, misleading copy.

Golden scenarios

A golden scenario is a deliberately chosen case with an expected product judgment written **before** looking at the model output. It prevents the model from quietly redefining what "good" means.

Scenario shape	What to record
Given	Current State + relevant Rules + incoming Evidence + open Questions/Reviews when applicable.
Expected interpretation	Review / no Review, affected State, exact proposal or explicit uncertainty.
Prohibited behavior	What the model or software must not infer, mutate, or hide.
After human action	Expected Current State, History, Question/Review lifecycle, and downstream Ask/Project behavior.

Important: A plausible model answer can still be wrong for the product. "Valid JSON" only proves shape; it does not prove correct judgment.

Testing & Evaluating Probabilistic Systems

Tests, evals, golden scenarios, regressions, and how to separate model failures from product or software failures.

Turn every meaningful defect into permanent coverage

When QA finds a real bug, preserve the exact case. Add the smallest automated test or eval that would have caught it, then add nearby cases so the fix does not overfit.

Case type	Example for State	Why it matters
Original failure	"Password reset tickets were approved for automation."	Confirms approval is preserved without inventing deployment.
Paraphrase	"Security signed off on automating password resets."	Checks the fix is semantic, not tied to one phrase.
Positive neighbor	"Password reset automation is live in production."	Ensures real implementation evidence can establish implementation.
Negative neighbor	"We may automate password resets later."	Speculation should not become approved/current truth.
Failure / uncertainty	Provider times out after Evidence is saved.	Evidence remains durable; retry analysis without duplicating the Note.

Useful evaluation dimensions

- **State integrity:** Was authoritative State preserved correctly?
- **Interpretation:** Did the model identify the right consequence, State items, and proposal?
- **User burden:** Did the system create unnecessary Reviews or blockers?
- **Groundedness:** Did the output stay within what the Evidence actually established?
- **Failure behavior:** Did timeouts, malformed output, stale data, and retries degrade safely?
- **Consistency:** Does behavior remain acceptable across paraphrases and repeated runs?

When an eval fails

Ask in order: **1)** Is the expected behavior actually specified? **2)** Is the model wrong? **3)** Is validation/application logic wrong? **4)** Is the UI presenting the right underlying behavior badly?

One-sentence summary

Use deterministic tests for invariants, evals for judgment, and real QA to discover the cases neither of them imagined.